RUBINA DANGOL MAHARJAN
AI FELLOWSHIP
FUSEMACHINE

10th May 2026

WeeK-2
Task-2

Build a Customer API

Presented to

Prabhat Ale

FuseMachine

Presented by

Rubina Maharjan

TABLE OF CONTENTS

ABOUT

Assignment Overview

This assignment focuses on building a **Customer API** that connects users with customer-related data stored in a PostgreSQL database. The main goal is to create an API that can safely and efficiently fetch, create, update, and delete customer information from the database. The assignment also emphasizes the use of **layered architecture**, where the application is divided into separate files, each with a specific responsibility. This makes the project cleaner, easier to manage, and more professional.

Database name: mydb

Goal: The system should be able to:

- Connect to a PostgreSQL database
- Validate user input using schemas
- Perform CRUD operations
- Handle API routes properly
- Manage errors gracefully
- Add logging for better debugging and monitoring
- Follow professional software development practices

Files:

- **main.py**: creates the FastAPI app, includes routers, and initializes tables on startup.
- **database.py**: loads DATABASE_URL, creates the SQLAlchemy engine/session, and provides get_db().
- **models.py**: SQLAlchemy ORM models that map to the database tables.
- **schemas.py**: Pydantic models for input validation and response shapes.
- **crud.py**: database operations (create/read/update/delete) using SQLAlchemy.
- **customers.py**: HTTP endpoints for customers, orders, and payments.
- **logger.py**: shared logging configuration (file + console).
- **__init__.py**: marks the routers package.
- **docker-compose.yml**: defines the PostgreSQL container and volume.
- **seed.sql**: creates tables and inserts sample data.
- **.env**: stores the database connection string.
- **requirements.txt**: Python dependencies.

System Architecture

4-layer clean architecture

Layer 1: Connection Layer (database.py)

1. Creating the database connection
2. Opening and closing sessions
3. Reading database credentials from the .env file
4. Keeping sensitive data such as database passwords outside the source code

Layer 2: Data Format Layer: schemas.py

The schemas.py file contains **Pydantic models**. These models define the structure and data types of the request and response data.

For example, the customer schema may define:

- customerNumber as an integer
- customerName as a string
- Phone number, address, city, country, and other customer fields with proper data types

This layer helps validate data before it reaches the database. If a user sends invalid data, such as a number instead of a name, the schema catches the error early.

Important schemas include:

- CustomerCreate
- CustomerOut
- CustomerUpdate

CustomerCreate is used when adding a new customer, CustomerOut is used when returning customer information, and CustomerUpdate is used when updating existing customer details.

Layer 3: Logic Layer: crud.py

The crud.py file contains the main database operations. CRUD stands for:

- **Create**
- **Read**
- **Update**
- **Delete**

This file communicates directly with the database using **SQLAlchemy ORM**. It should not handle HTTP requests or user interaction directly. Its only job is to perform database queries and return results.

Layer 4: API Layer: router.py

The router.py file is responsible for handling API endpoints. It receives HTTP requests from users, calls the correct CRUD function, and returns the response.

This layer acts as the front desk of the application because it directly communicates with the users.

API Behavior

The API should not only work correctly but also handle errors properly.

Get Customer by ID

When a user requests a specific customer, the API should:

1. Search the database using the given customer ID.
2. Return the customer data if the customer exists.
3. Return a **404 Not Found** error if the customer does not exist.
4. This prevents the application from crashing and gives users a clear message.

Example:

```
INFO:     127.0.0.1:50541 - "GET /customers/9999 HTTP/1.1" 404 Not Found
```

If customer 9999 exists, the API returns customer details.

If the customer does not exist, it returns:

```
{
  "detail": "Customer not found"
}
```

Pagination

Pagination is required when listing customers

If the database contains thousands of customers, returning all records at once is inefficient.

skip: how many records to skip

limit: how many records to return

Example:

```
INFO:     127.0.0.1:62554 - "GET /customers/?skip=0&limit=10 HTTP/1.1" 200 OK
```

This improves performance and makes the API easier to use.

Related Data

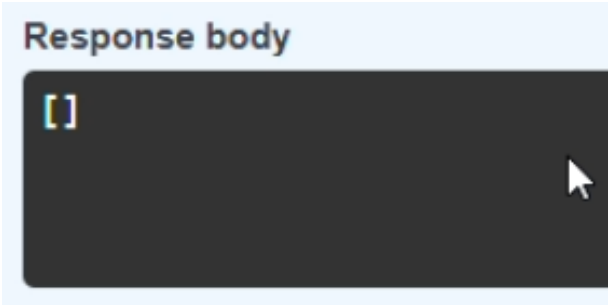
Customers' related data

Customers may have related data such as:

- Orders
- Payments

The API should be able to return this information using separate endpoints. If a customer has no orders or payments, the API should return an empty list instead of producing an error.

Example:



This makes the API stable and user-friendly.

Logging

Helps track what is happening inside the application

Logging is added in different layers:

In database.py

- Log database connection success
- Log database connection failure
- Log connection closing events

In schemas.py

- Log validation errors

In crud.py

- Log create, read, update, and delete operations
- Log missing records
- Log failed queries

In router.py

- Log incoming API requests
- Log responses
- Log endpoint errors such as 404 or 500 errors

The assignment uses Python's built-in logging module. A central logger.py file is also created so that logging remains consistent across the project.

Reflection

Twelve-Factor App Principles Used

Factor II: Dependencies

The project should use a requirements.txt file and a virtual environment. This ensures that the same library versions are used when the project is shared or deployed.

Factor IV: Backing Services

PostgreSQL is treated as a backing service. Since SQLAlchemy ORM is used, the application can be changed to use another database later with fewer changes in the code.

Factor III: Config Management

Database credentials and configuration should be stored in a .env file instead of being written directly in the code. This protects sensitive information and makes the application easier to configure.

Error journal

Error	Cause	Solution
port is already allocated	Port 5432 already used by another PostgreSQL instance	Stop the other Postgres service or change docker-compose to map 5433:5432 and update DATABASE_URL.
pg_config executable not found when installing psycopg2	psycopg2 was trying to build from source	Use a supported Python (3.12) and psycopg2-binary package.
no pq wrapper available (psycopg)	psycopg v3 couldn't load binary/libpq on Windows	Switch to psycopg2-binary and use postgresql+psycopg2 in DATABASE_URL.
ModuleNotFoundError: pydantic_core._pydantic_core	Broken or incomplete dependency install	Upgrade pip, reinstall pydantic packages, then reinstall requirements.

Testing the API

After completing the implementation, the API can be tested using FastAPI's automatic documentation.

The documentation is available at:

```
http://localhost:8000/docs
```

From this page, users can test all endpoints directly from the browser.

Conclusion

This assignment provides practical experience in building a professional backend API using FastAPI, PostgreSQL, SQLAlchemy, and Pydantic. It teaches important backend development concepts such as database connection, schema validation, CRUD operations, routing, pagination, error handling, logging, and environment-based configuration.

Overall, the assignment helps students understand how real-world APIs are structured and maintained. By following layered architecture, the project becomes cleaner, more secure, easier to debug, and easier to expand in the future.